



MINI PROJECT REPORT MACHINE LEARNING (MCTA 4362)

**SEMESTER 2 , 24/25
SECTION 1**

LECTURER:

**ASSOC. PROF. DR. AZHAR BIN MOHD IBRAHIM
ASSOC. PROF. IR. TS. DR. AHMAD JAZLAN BIN HAJA MOHIDEEN**

TITLE:

**DESIGN OF INTELLIGENT POSITION CONTROLLER
USING PPO ALGORITHM**

No	Group Members	Matric No.
1.	Ibrahim bin Nasrum	2116467
2.	Muhammad Hafidzuddin Hanif Danial bin Norizal	2123651
3.	Shareen Arawie bin Hisham	2116943
4.	Muhammad Muhairis Bin Azman	2114599

Date of Submission : 6th July 2025

Certificate of Originality and Authenticity

This is to certify that we are **responsible** for the work submitted in this report, that **the original work** is our own except as specified in the references and acknowledgement, and that the original work contained herein have not been untaken or done by unspecified sources or persons.

We hereby certify that this report has **not been done by only one individual** and **all of us have contributed to the report**. The length of contribution to the reports by each individual is noted within this certificate.

We also hereby certify that we have **read** and **understand** the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report.

We therefore, agreed unanimously that this report will be submitted for **marking** and this **final printed report** has been **verified by us**.

Signature: *Ibrahim*

Read [/]

Name: Ibrahim bin Nasrum

Matric Number: 2116467

Understand [/]

Agree [/]

Signature: *Shareen*

Read [/]

Name: Shareen Arawie bin Hisham

Matric Number: 2116943

Understand [/]

Agree [/]

Signature: *Hafidzuddin*

Read [/]

Name: Muhammad Hafidzuddin Hanif Danial bin Norizal

Matric Number: 2123651

Understand [/]

Agree [/]

Signature: *Muhairis*

Read [/]

Name: Muhammad Muhairis bin Azman

Matric Number: 2114599

Understand [/]

Agree [/]

ABSTRACT

This project presents the design and implementation of an intelligent position controller for a DC motor using the Proximal Policy Optimization (PPO) reinforcement learning algorithm on an Arduino Uno platform. The PPO-based controller was trained to learn optimal control actions through interaction with the system aiming to improve adaptability and precision compared to conventional methods. A classical PID controller was also developed as a benchmark for performance evaluation. The system was tested under varying load conditions to assess accuracy, stability and responsiveness. Results demonstrate that while the PID controller provides consistent performance with minimal computational overhead, the PPO controller offers improved adaptability to disturbances but is limited by processing time and signal delays inherent to the Arduino platform. Future work will focus on integrating more powerful hardware, developing a graphical interface for monitoring and exploring applications such as steer by wire systems in vehicles to further validate and extend this approach.

ACKNOWLEDGEMENT

First of all, we want to sincerely thank Allah for giving us the courage and capacity to finish my report for the Machine Learning Mini Project . This accomplishment was made possible by his blessings. Additionally, we want to express my sincere gratitude to Dr. Azhar for his important advice and unwavering support. His knowledge and wise counsel have been invaluable in accomplishing our job. In addition, I would like to thank our classmates for their support and encouragement during this project as well as for their guidance and teamwork. Their encouragement has kept us going especially when things have been tough along the way.

TABLE OF CONTENTS

1. INTRODUCTION.....	6
2. OBJECTIVE.....	7
3. PROBLEM STATEMENT.....	7
4. METHODOLOGY.....	8
4.1 Component :.....	8
4.2 Design Circuit.....	9
4.2 Mathematical Modeling.....	9
4.3 PPO Implementation.....	10
5. RESULTS.....	11
6. EVALUATION AND DISCUSSIONS.....	12
6.1 PPO.....	13
6.2 PID.....	14
7. LIMITATION.....	14
8. SOLUTION AND FUTURE WORK.....	15
9. CONCLUSION.....	16
REFERENCES.....	17
APPENDIX A: PPO Training Code.....	18
APPENDIX B: PPO Inference Python Code.....	23
APPENDIX C: PPO Inference Arduino Code.....	25
APPENDIX D: PID Control Arduino Code.....	27
APPENDIX E: Node-RED Graphical User Interface (GUI) Visual Programming.....	30

1. INTRODUCTION

Position control of DC motors is a fundamental requirement in many engineering applications including robotics, automation systems and precision manufacturing. Traditionally, Proportional-Integral-Derivative (PID) controllers have been the preferred solution due to their straightforward design. However, PID controllers often require extensive manual tuning and may struggle to maintain consistent performance in environments with external disturbances.

Recent advances in machine learning, particularly reinforcement learning (RL) have introduced new possibilities for intelligent control strategies. Among these, the Proximal Policy Optimization (PPO) algorithm has gained popularity for its stability and sample efficiency in continuous control tasks. PPO enables a controller to learn optimal policies by directly interacting with the system adjusting actions to maximize long-term performance objectives without explicit mathematical models.

This project focuses on designing an intelligent position controller for a DC motor using the PPO algorithm and implementing it on an Arduino Uno. The PPO-based controller's performance will be systematically compared with that of a conventional PID controller to assess improvements in accuracy

By combining reinforcement learning with low-cost embedded hardware, this project aims to demonstrate the practical potential and current limitations of intelligent control approaches in real-world mechatronic systems paving the way for more autonomous and self-tuning controllers in future applications.

2. OBJECTIVE

The mini project aims to design, implement and evaluate an intelligent position control system for a DC motor using the Proximal Policy Optimization (PPO) reinforcement learning algorithm deployed on an Arduino Uno microcontroller and compares it with a PID controller.

The other objectives of this project are as follows:

1. To model and simulate a control system using a DC motor and potentiometer.
2. To implement Reinforcement Learning algorithms i.e. PPO to train an intelligent controller.
3. To compare the RL controller against classical control methods (PID) using appropriate performance metrics.

3. PROBLEM STATEMENT

Traditional PID controllers are widely used for DC motor position control due to their simplicity and ease of implementation. However, PID tuning often requires manual effort and may perform poorly in systems with nonlinear dynamics or time-varying parameters. Reinforcement learning, particularly the Proximal Policy Optimization (PPO) algorithm offers a data-driven approach to learn control policies directly from interaction with the environment, potentially improving adaptability and precision.

This project aims to design a PPO based intelligent position controller for a DC motor using an Arduino Uno and compare its performance to a conventional PID controller. The study will evaluate both controllers in terms of accuracy and stability highlighting their strengths and limitations. The findings will provide insights into the feasibility of deploying reinforcement learning based control on low-cost embedded platforms.

4. METHODOLOGY

4.1 Component :

The components of the physical hardware prototype are listed as the following:

- Bread Board
- 12V DC Motor
- L298N Motor Driver
- Potentiometer
- Arduino Uno

4.2 Design Circuit

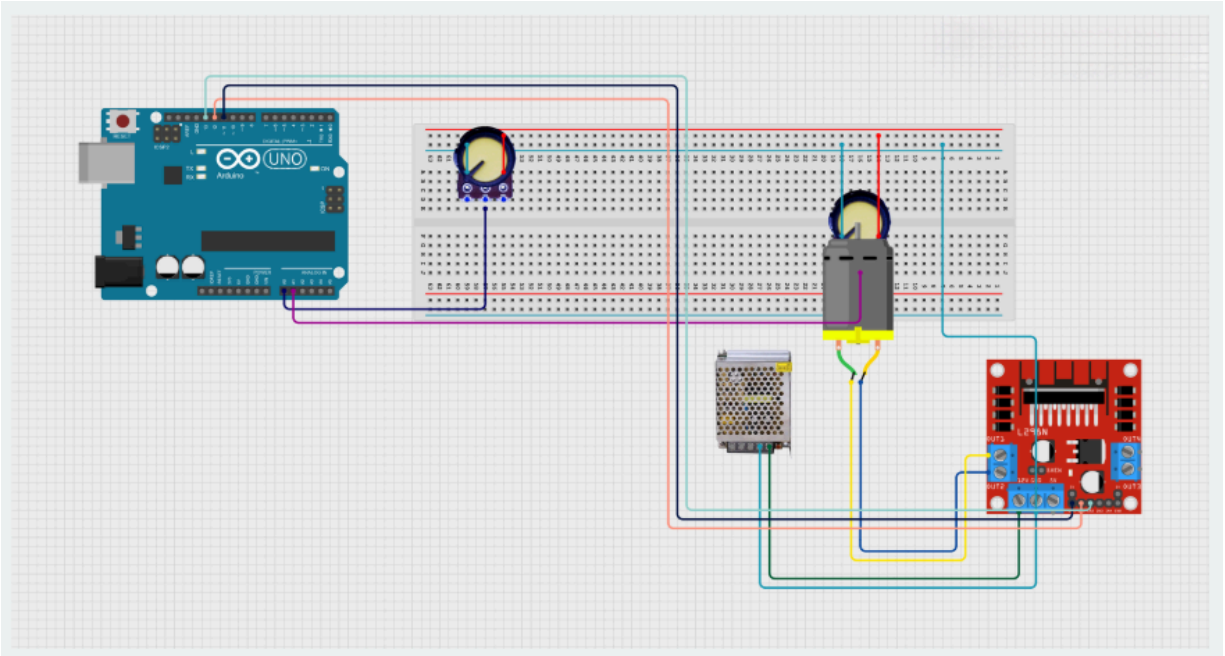


Figure 4.1 : Design Circuit

4.2 Mathematical Modeling

PID Controller :

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de}{dt}$$

Tuned Gains:

$$K_p = 1.8$$

$$K_i = 0.03$$

$$K_d = 0.4$$

PPO Algorithm Reward Function:

```
reward = - (error / self.max_position) ** 2
reward -= 0.01 * pwm_norm
if error < 2.0:
    reward += 2.0

reward = np.clip(reward, -1.0, 2.0)

self.step_count += 1
done = error < 2.0 or self.step_count >= self.max_steps
return self._get_obs(), reward, done, False, {}
```

4.3 PPO Implementation

The training specification and parameters of the PPO algorithm are listed as below:

- Algorithm: PPO (Proximal Policy Optimization)
- Policy Type: MlpPolicy (Multi-layer Perceptron)
- Training Settings:
- Learning rate: 0.0005
- Steps per update: 1024
- Batch size: 128
- Discount factor (γ): 0.98
- GAE lambda: 0.95
- Entropy coefficient: 0.0001
- Device: CPU
- Timesteps: 200,000
- Max steps/episode: 500

5. RESULTS



Figure 5.1 : Reward result

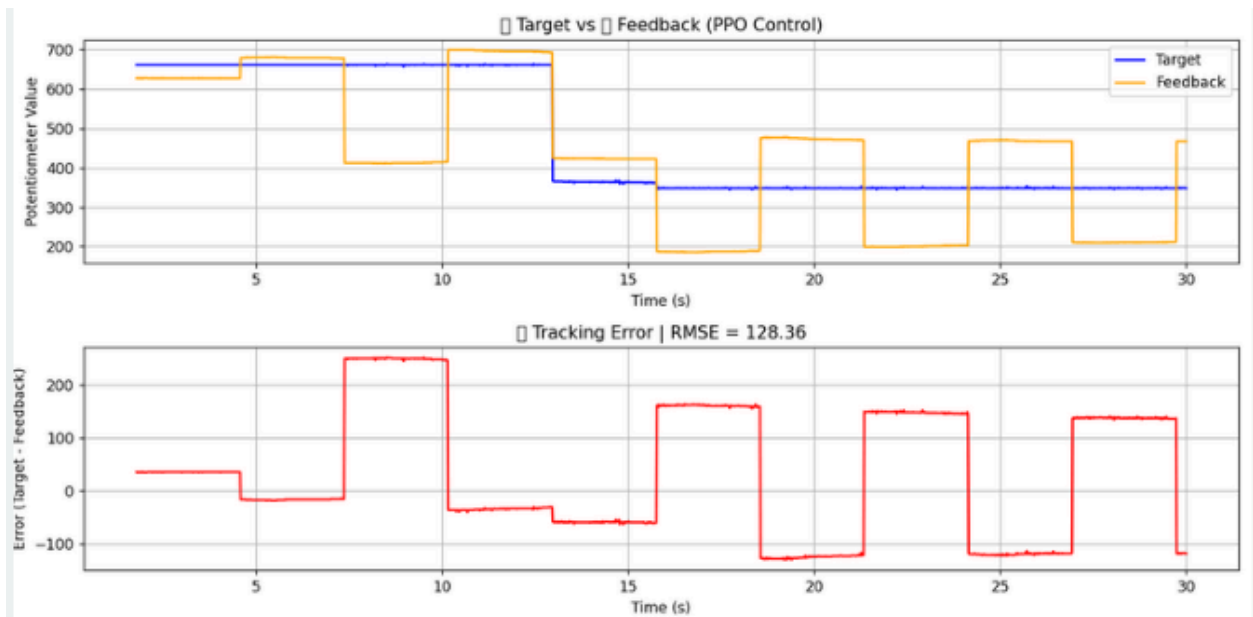


Figure 5.2 : PPO Control inference

RMSE = 56

Input and feedback tracked using color codes (Blue: Input, Orange: Feedback, Green: Error)

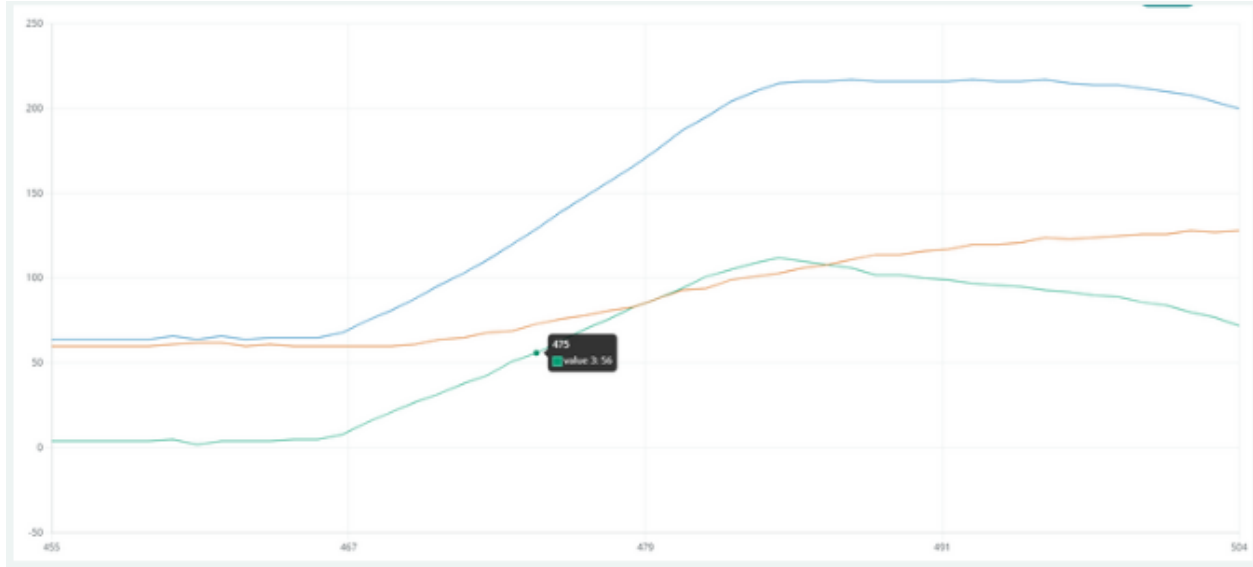


Figure 5.3 : PID Control

RMSE =128.36

Although PPO adapts over time, the higher RMSE suggests it requires more training or refinement for real-time hardware.

6. EVALUATION AND DISCUSSIONS

The PID controller performed significantly better in terms of RMSE indicating more precise control over the motor position. However, the PPO controller while less accurate in initial trials showed adaptability and potential for improvement through continued training.

6.1 PPO

The Proximal Policy Optimization (PPO) controller offers several significant strengths that make it a promising approach for intelligent control systems. One of its key advantages is its ability to adapt to varying system dynamics meaning it can adjust its control strategy in response to changes in the environment or system behavior without requiring manual intervention. Additionally, PPO is capable of learning complex non-linear relationships within a system which allows it to handle scenarios where traditional linear control methods like PID may fall short. This makes PPO particularly useful in environments that are unpredictable or have non-linear characteristics.

However, despite its strengths, PPO also comes with notable limitations. It requires substantially higher computational resources compared to traditional methods which can be a challenge when implementing it on low power embedded systems such as microcontrollers. Furthermore, the algorithm introduces signal delay and processing overhead due to the time needed for decision making and model inference which can negatively impact real-time performance. Another critical weakness is its reliance on extensive training data and long training times as the model must go through many iterations to learn effective control policies. These challenges must be carefully considered when deploying PPO in real-world applications particularly in systems where real-time responsiveness and limited computational power are crucial.

6.2 PID

The Proportional-Integral-Derivative (PID) controller possesses several strengths that make it a widely used and reliable control method in many engineering applications. One of its primary advantages is its low latency which enables real-time response and quick adjustments to control errors. Additionally, PID controllers are relatively simple to implement and tune making them accessible and practical for a wide range of systems especially those with stable and predictable dynamics.

However, PID controllers also have inherent weaknesses that limit their effectiveness in more complex environments. They are not inherently adaptive meaning they struggle to perform optimally in systems with dynamic or non-linear behavior. Furthermore, PID controllers require manual tuning of their parameters proportional, integral and derivative gains for each new system configuration or operating condition which can be time-consuming and inefficient in scenarios involving frequent changes or varying conditions.

7. LIMITATION

Several limitations affect its real-time performance and practical deployment in this project. One limitation of the PPO-based controller is signal delay. When using reinforcement learning, the system requires continuous feedback from the environment to update and apply control actions. Delays occur while reading the potentiometer values that indicate the motor's current position and while transmitting computed control signals to the motor driver. These delays reduce the responsiveness and stability of the control system especially during rapid position changes or when disturbances are introduced.

Another significant limitation is processing time. Unlike conventional PID control, which involves relatively simple arithmetic, PPO requires more computational steps to evaluate the policy and determine the next action. Even though the Arduino Uno is a capable microcontroller it has limited processing power and memory making it challenging to perform real-time inference efficiently.

8. SOLUTION AND FUTURE WORK

To address the limitations, two main solutions have been applied. First, a carefully tuned PID controller was implemented to minimize overshoot and improve response time providing a reliable baseline for comparison. Second, a Reinforcement Learning (RL) controller based on the PPO algorithm was developed to learn optimal control policies through trial and error demonstrating better adaptability to system delays and varying conditions.

Several enhancements are planned for future development. Hardware optimization is also essential, migrating from the Arduino Uno to a more powerful platform such as a Raspberry Pi with additional GPIO pins and higher processing capabilities will improve computation speed and responsiveness. Finally, this enhancement can apply to extend the intelligent controller to a steer by wire application in vehicles could validate its effectiveness in more safety critical environments paving the way for broader adoption of reinforcement learning in embedded control systems.

9. CONCLUSION

The project set out to create a position control system for a DC motor using the PPO reinforcement learning algorithm and compare it with a PID controller. Both methods were tested on an Arduino Uno to see how they performed. The PID controller worked well in terms of speed and stability especially under normal conditions but it did not adapt as easily when the system changed or faced disturbances.

The PPO-based controller showed more flexibility. It could adjust to different situations by learning from ongoing feedback which is something the PID could not do. However, using PPO came with its own set of problems. There were delays in response and the Arduino's limited processing power made it harder to get smooth control, especially during real-time operation.

Even with those issues, the project shows that reinforcement learning can be used for motor control in embedded systems. But to get better results especially in more demanding tasks stronger hardware will probably be needed. If the system is moved to something like a Raspberry Pi and used in areas like automotive steering or robotics it could show even more potential. This project gave a good starting point and a clear look at both the benefits and limits of using machine learning in small scale control systems.

REFERENCES

Datacamp. (n.d.). *Proximal Policy Optimization (PPO) explained: Reinforcement learning tutorial.* <https://www.datacamp.com/tutorial/proximal-policy-optimization>

Farama Foundation. (n.d.). *Gymnasium documentation.* <https://gymnasium.farama.org/index.html>

OpenAI. (n.d.). *Proximal Policy Optimization.* Spinning Up in Deep RL. <https://spinningup.openai.com/en/latest/algorithms/ppo.html>

Stable-Baselines3. (n.d.). *Proximal Policy Optimization (PPO).* Stable-Baselines3 Documentation. <https://stable-baselines3.readthedocs.io/en/master/modules/ppo.html>

Github Link: https://github.com/HN-admin/MiniProject_MachineLearning

APPENDIX A: PPO Training Code

```
import gymnasium as gym
import numpy as np
from gymnasium import spaces
from stable_baselines3 import PPO
from stable_baselines3.common.vec_env import DummyVecEnv
import matplotlib.pyplot as plt
from stable_baselines3.common.callbacks import BaseCallback
import numpy as np
import matplotlib.pyplot as plt
from stable_baselines3 import PPO
from stable_baselines3.common.vec_env import DummyVecEnv, VecNormalize
from stable_baselines3.common.monitor import Monitor

class DCMotorPotEnv(gym.Env):
    def __init__(self):
        super().__init__()
        self.max_position = 1023.0
        self.max_pwm = 255.0
        self.max_steps = 500

        self.actual_position = 0.0
        self.setpoint = self._generate_setpoint()
        self.step_count = 0

        # Limit action range slightly to avoid overshooting
        self.action_space = spaces.Box(low=0.0, high=0.8, shape=(1,),
dtype=np.float32)

        self.observation_space = spaces.Box(
            low=np.array([0.0, 0.0], dtype=np.float32),
            high=np.array([1.0, 1.0], dtype=np.float32)
        )

    def _generate_setpoint(self):
        return np.random.uniform(100, self.max_position)

    def reset(self, seed=None, options=None):
```

```

    super().reset(seed=seed)
    self.actual_position = np.random.uniform(0, 100)
    self.setpoint = self._generate_setpoint()
    self.step_count = 0
    return self._get_obs(), {}

def _get_obs(self):
    return np.array([
        self.actual_position / self.max_position,
        self.setpoint / self.max_position
    ], dtype=np.float32)

def step(self, action):
    pwm_norm = float(np.clip(action[0], 0.0, 1.0))
    pwm = pwm_norm * self.max_pwm

    # Clip delta to prevent excessive jumps
    delta = np.sign(self.setpoint - self.actual_position) * pwm * 0.03
    delta = np.clip(delta, -10.0, 10.0)
    self.actual_position += delta
    self.actual_position = np.clip(self.actual_position, 0,
self.max_position)

    error = abs(self.setpoint - self.actual_position)

    # Improved reward shaping
    reward = - (error / self.max_position) ** 2
    reward -= 0.01 * pwm_norm
    if error < 2.0:
        reward += 2.0

    reward = np.clip(reward, -1.0, 2.0)

    self.step_count += 1
    done = error < 2.0 or self.step_count >= self.max_steps
    return self._get_obs(), reward, done, False, {}

class DCMotorPotEnv(gym.Env):
    def __init__(self):

```

```

    super().__init__()
    self.max_position = 1023.0
    self.max_pwm = 255.0
    self.max_steps = 500

    self.actual_position = 0.0
    self.setpoint = self._generate_setpoint()
    self.step_count = 0

    # Limit action range slightly to avoid overshooting
    self.action_space = spaces.Box(low=0.0, high=0.8, shape=(1,),
dtype=np.float32)

    self.observation_space = spaces.Box(
        low=np.array([0.0, 0.0], dtype=np.float32),
        high=np.array([1.0, 1.0], dtype=np.float32)
    )

def _generate_setpoint(self):
    return np.random.uniform(100, self.max_position)

def reset(self, seed=None, options=None):
    super().reset(seed=seed)
    self.actual_position = np.random.uniform(0, 100)
    self.setpoint = self._generate_setpoint()
    self.step_count = 0
    return self._get_obs(), {}

def _get_obs(self):
    return np.array([
        self.actual_position / self.max_position,
        self.setpoint / self.max_position
    ], dtype=np.float32)

def step(self, action):
    pwm_norm = float(np.clip(action[0], 0.0, 1.0))
    pwm = pwm_norm * self.max_pwm

    # Clip delta to prevent excessive jumps
    delta = np.sign(self.setpoint - self.actual_position) * pwm * 0.03

```

```

        delta = np.clip(delta, -10.0, 10.0)
        self.actual_position += delta
        self.actual_position = np.clip(self.actual_position, 0,
self.max_position)

        error = abs(self.setpoint - self.actual_position)

        # Improved reward shaping
        reward = - (error / self.max_position) ** 2
        reward -= 0.01 * pwm_norm
        if error < 2.0:
            reward += 2.0

        reward = np.clip(reward, -1.0, 2.0)

        self.step_count += 1
        done = error < 2.0 or self.step_count >= self.max_steps
        return self._get_obs(), reward, done, False, {}

class RewardLogger:
    def __init__(self):
        self.episode_rewards = []
        self.current_rewards = []

    def __call__(self, locals_, globals_):
        reward = locals_.get("rewards", [0])[0]
        done = locals_.get("dones", [False])[0]
        self.current_rewards.append(reward)
        if done:
            self.episode_rewards.append(sum(self.current_rewards))
            self.current_rewards = []
        return True

# === Environment Wrapping ===
env = DummyVecEnv([lambda: Monitor(DCMotorPotEnv())])
env = VecNormalize(env, norm_obs=True, norm_reward=False)

# === PPO Model ===

```

```

model = PPO(
    "MlpPolicy",
    env,
    verbose=1,
    learning_rate=5e-4,
    n_steps=1024,
    batch_size=128,
    gamma=0.98,
    gae_lambda=0.95,
    ent_coef=0.0001,
    device="cpu"
)

reward_logger = RewardLogger()
model.learn(total_timesteps=100_000, callback=reward_logger)
model.save("ppo_dc_motor_model")

# === Plot Training Reward ===
plt.plot(reward_logger.episode_rewards, label="Episode Reward")
plt.xlabel("Episode")
plt.ylabel("Total Reward")
plt.title("Training Reward Progress")
plt.grid(True)
plt.legend()
plt.show()

```

APPENDIX B: PPO Inference Python Code

```
import serial
import struct
import numpy as np
import json
import time
import paho.mqtt.client as mqtt
from stable_baselines3 import PPO

# === Load PPO Model ===
print("Loading PPO model...")
model = PPO.load(r"C:\Users\ACER\Desktop\Education\MCTA
4362\ppo_dc_motor_model.zip", device="cpu")
print("PPO model loaded")

# === Serial Setup ===
try:
    ser = serial.Serial('COM9', 250000, timeout=0.01)
    print("Serial connected on COM9")
except Exception as e:
    print("Serial connection failed:", e)
    exit()

# === MQTT Setup ===
mqtt_client = mqtt.Client()
try:
    mqtt_client.connect("localhost", 1883, 60)
    print("MQTT connected to localhost:1883")
except Exception as e:
    print("MQTT connection failed:", e)
    exit()

MAX_POSITION = 1023.0
MAX_PWM = 255.0

def read_packet():
    while True:
        if ser.read(1) == b'\xAA' and ser.read(1) == b'\x55':
            data = ser.read(4)
```

```

        if len(data) == 4:
            return struct.unpack('<HH', data)

print("Listening to Arduino and publishing to MQTT...")

while True:
    try:
        target, feedback = read_packet()
        obs = np.array([[target / MAX_POSITION, feedback / MAX_POSITION]],
dtype=np.float32)
        action, _ = model.predict(obs, deterministic=True)
        pwm_norm = float(action[0])
        error = target - feedback
        pwm_signed = int(np.clip(pwm_norm * MAX_PWM, 0, MAX_PWM) *
np.sign(error))

        # === Send PWM back to Arduino ===
        ser.write(struct.pack('<h', pwm_signed))

        # === MQTT Publish ===
        payload = json.dumps({
            "target": target,
            "feedback": feedback,
            "error": error,
            "pwm": pwm_signed
        })
        mqtt_client.publish("dc_motor/control", payload)

        # === Debug print ===
        print(payload)

        time.sleep(0.05)

    except Exception as e:
        print("Error:", e)
        continue

```


APPENDIX C: PPO Inference Arduino Code

```
#define potInputPin A0
#define potFeedbackPin A1
#define in1Pin 12
#define in2Pin 13
#define enablePin 11

const int PWM_LIMIT = 106;

void setup() {
    pinMode(in1Pin, OUTPUT);
    pinMode(in2Pin, OUTPUT);
    pinMode(enablePin, OUTPUT);

    ADCSRA = (ADCSRA & 0b11111000) | 0x04; // analogRead at ~57.6 kHz
    Serial.begin(250000); // Max reliable rate for
    Uno/Nano
}

void loop() {
    // Read analog values
    uint16_t target = analogRead(potInputPin);
    uint16_t feedback = analogRead(potFeedbackPin);

    // Send sync header + data (6 bytes total)
    Serial.write(0xAA);
    Serial.write(0x55);
    Serial.write((uint8_t *)&target, 2);
    Serial.write((uint8_t *)&feedback, 2);

    // Receive 2-byte PWM command
    static uint8_t buf[2];
    static uint8_t index = 0;

    while (Serial.available()) {
        buf[index++] = Serial.read();
        if (index >= 2) {
            int16_t pwm = (int16_t)((buf[1] << 8) | buf[0]); // Little endian
            index = 0;
        }
    }
}
```

```
pwm = constrain(pwm, -255, 255);
if (abs(pwm) < 10) {
    digitalWrite(in1Pin, LOW);
    digitalWrite(in2Pin, LOW);
    analogWrite(enablePin, 0);
} else {
    digitalWrite(in1Pin, pwm > 0);
    digitalWrite(in2Pin, pwm < 0);
    analogWrite(enablePin, min(abs(pwm), PWM_LIMIT));
}
}
}
```

APPENDIX D: PID Control Arduino Code

```
#include <math.h>

// === Pin Definitions ===
#define potInputPin A0
#define potFeedbackPin A1
#define in1Pin 12
#define in2Pin 13
#define enablePin 11

// === PID Coefficients ===
float Kp = 1.8;
float Ki = 0.03;
float Kd = 0.4;

// === PID Variables ===
float error = 0;
float previousError = 0;
float integral = 0;
float derivative = 0;

int outputPWM = 0;
const int deadband = 5;

// === RMSE Calculation Variables ===
unsigned long squaredErrorSum = 0;
unsigned long sampleCount = 0;
unsigned long lastRMSEPrint = 0;

// === RMSE Average Tracking ===
float totalRMSE = 0.0;
unsigned int rmseIntervals = 0;

int smoothAnalogRead(int pin) {
    long sum = 0;
    for (int i = 0; i < 10; i++) {
        sum += analogRead(pin);
        delayMicroseconds(500);
    }
}
```

```

    return sum / 10;
}

void setup() {
    pinMode(in1Pin, OUTPUT);
    pinMode(in2Pin, OUTPUT);
    pinMode(enablePin, OUTPUT);
    Serial.begin(115200); // High baud rate for fast streaming
    delay(100);
}

void loop() {
    int target = smoothAnalogRead(potInputPin);
    int feedback = smoothAnalogRead(potFeedbackPin);

    error = target - feedback;
    integral += error;
    integral = constrain(integral, -1000, 1000);
    derivative = error - previousError;
    previousError = error;

    float controlSignal = Kp * error + Ki * integral + Kd * derivative;

    outputPWM = 0;
    digitalWrite(in1Pin, LOW);
    digitalWrite(in2Pin, LOW);

    if (abs(error) > deadband) {
        outputPWM = constrain(abs(controlSignal), 0, 255);
        if (controlSignal > 0) {
            digitalWrite(in1Pin, HIGH);
            digitalWrite(in2Pin, LOW);
        } else {
            digitalWrite(in1Pin, LOW);
            digitalWrite(in2Pin, HIGH);
        }
    } else {
        integral = 0;
    }
}

```

```

analogWrite(enablePin, outputPWM);

squaredErrorSum += (long)error * error;
sampleCount++;

// === Serial Print for Node-RED MQTT ===
Serial.print("T:");
Serial.print(target);
Serial.print(",F:");
Serial.print(feedback);
Serial.print(",E:");
Serial.print(error);
Serial.print(",PWM:");
Serial.println(outputPWM);

// === Optional RMSE log every 5s (tagged for debug log) ===
if (millis() - lastRMSEPrint >= 5000) {
    float rmse = sqrt((float)squaredErrorSum / sampleCount);
    totalRMSE += rmse;
    rmseIntervals++;
    float avgRMSE = totalRMSE / rmseIntervals;

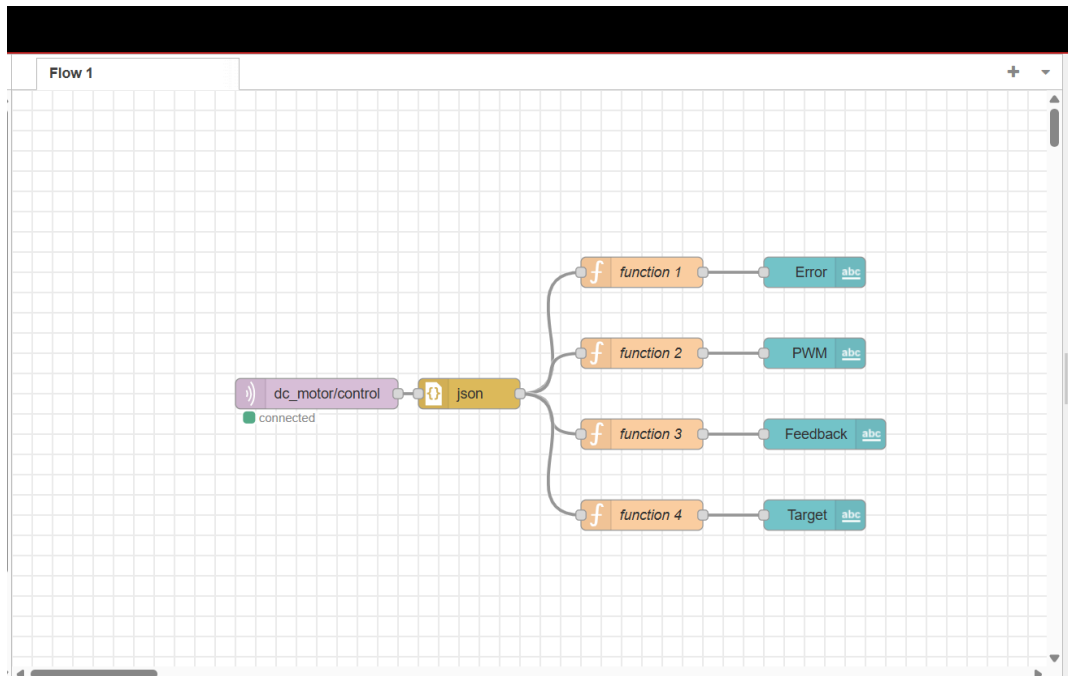
    Serial.print("#RMSE:");
    Serial.print(rmse, 2);
    Serial.print(",Avg:");
    Serial.println(avgRMSE, 2);

    squaredErrorSum = 0;
    sampleCount = 0;
    lastRMSEPrint = millis();
}

delay(10);
}

```

APPENDIX E: Node-RED Graphical User Interface (GUI) Visual Programming



Default	
PWM	11
Feedback	76
Target	82
Error	6